Curs 9

28.11.2007

Sumar

- ✓ Funcţii
  - Definire
  - Declarare
  - Apel
  - Parametri formali
  - Parametri efectivi
  - Transmitere prin valoare, transmitere prin adresă.
  - Recursivitate
  - *Linkage*

8. Funcţii

Funcţii

- ✓ Atunci când se scriu programe complexe este necesară utilizarea unităţilor de program care în C sunt referite ca funcţii.
- ✓ Ele reprezintă **prelucrări unitare** care pot fi utilizate şi în alte programe.
- ✓ **Funcţiile sunt specifice programării structurate.**
- ✓ Pot fi *standard* sau *definite de utilizator*.

Funcţii

- ✓ Un program în limbaj C este format dintr-un program principal în care sunt definite funcţii care se apelează din funcţia *main()* sau unele din altele.
- ✓ În momentul când se apelează o funcţie, programul va continua cu executarea instrucţiunilor din funcţia respectivă, iar la finalul acesteia se va reveni la instrucţiunea care urmează după apel în funcţia în care s-a făcut apelul.

Atunci când apelează o funcţie se realizează următoarele acţiuni:

- ✓ Se întrerupe execuţia instrucţiunilor din unitatea apelantă.
- ✓ Se realizează corespondenţa între numele parametrilor formali şi valorile sau adresele argumentelor.
- ✓ Se creează variabile locale funcţiei cu rezervarea locaţiilor de memorie.
- ✓ Se execută instrucţiunile din funcţie.
- ✓ Se desfiinţează variabilele locale.
- ✓ Se revine în unitatea apelantă.

Avantajele utilizării funcţiilor

- Pentru operaţii care pot fi executate pentru **diferite variabile de acelaşi tip**, declararea funcţiilor care le descriu se face o singură dată, iar apelul ori de câte ori este nevoie.
- Se obţine un program **modularizat** - mai uşor de scris şi depanat.
- Funcţiile pot fi utilizate şi în **alte programe** decât cele în care au fost declarate; aceasta se poate realiza în C folosind *extern*.
- Funcţiile pot "ascunde" detalii ale operaţiilor din program (încapsulare), pe care nu este nevoie să le cunoaştem, **clarificând astfel întregul şi permiţând modificări mai uşoare**.

Funcţii standard

- ✓ Funcţii din **biblioteca standard de I/O**: scanf, printf.
- ✓ **Funcţii standard**: abs, sqrt, sin, cos, etc.

Funcţia *main()*

- ✓ **Funcţia** *main()* este strict necesară într-un program în C.
- ✓ Aceasta va returna către sistemul de operare fie valoarea 0, fie 1:
  - 0 – în cazul în care programul a fost executat cu succes;
  - 1 – programul s-a terminat cu eroare.
- ✓ *main()* nu poate returna *void* (v. ANSI C).

Funcţia *main()*

- ✓ *main()* poate fi declarată
  - fără parametri: int main()
  - cu doi parametri: int main (int argc, char *argv[]) – valorile argumentelor sunt furnizate de către sistemul de operare;
    - valoarea din argc reprezintă numărul de parametri din linia de comandă pe care o introducem de la tastatură
    - în tabelul indicat de argv se găsesc valorile parametrilor introduşi de la tastatură în linia de comandă (un tabel de *pointeri* către şirurile care reprezintă argumentele); argumentele sunt pasate programului de către *interpreterul* de linii de comandă al sistemului pe care rulează programul).

Funcţia *main()*

- argc este strict mai mare decât zero;
- argv[argc] este *null pointer;*
- argv[0] … argv[argc-1] sunt *pointeri* către şirurile al căror înţeles ulterior va fi precizat prin program;
- argv[0] este un şir care conţine numele programului sau un şir vid în cazul în care acesta nu este disponibil;
- celelalte elemente din argv reprezintă argumentele furnizate de către program.

mcua.c

maincua.c

Funcţia *main()*

- ✓ Pentru a ieşi din funcţia *main()* se poate folosi şi instrucţiunea *exit*:
  - Pentru terminare normală: exit(0) sau exit (EXIT_SUCCESS)
  - Pentru terminare anormală: exit(1) sau exit(EXIT_FAILURE).

Funcţii - definiţie, declaraţie, apel

Când sunt definite de utilizator trebuie declarate/definite şi apoi apelate.

- **Declaraţia** va conţine numele şi tipul funcţiei şi al parametrilor, fără a aloca spaţiu în memorie (**prototipul** funcţiei), iar **definiţia** conţine în plus şi descrierea datelor şi prelucrărilor pe care le realizează funcţia.
  - Avantajele prototipizării
    - creează cod mai structurat şi ca urmare mai uşor de citit;
    - permite compilatorului să verifice sintaxa apelurilor de funcţie.
- **Apelul** funcţiei se face printr-o instrucţiune de apel în care se indică numele funcţiei, iar dacă este cazul sunt precizate şi valorile curente, actuale ale parametrilor. Apelul are ca urmare activarea instrucţiunilor funcţiei.

Funcţii – definiţie, declaraţie

- ✓ Exemplu de declaraţie:

  int ***exemplu_declaratieF*** *(float arg1, int arg2);*

- ✓ Exemplu de definiţie

  long ***la_patrat****(int num) { return(num*num); }*

Sintaxa

- ✓ Sintaxa unei funcţii:

  tip_rez_functie *nume_fct* (<listă_par_form>); {bloc_instructiuni}
- ✓ *nume_fct* e folosit pentru apelul funcţiei.
- ✓ *lista parametrilor formali* – prin aceştia se transmit valori din funcţia apelantă în funcţia curentă.
- ✓ *bloc_instructiuni* descrie conţinutul funcţiei.

Definiţia unei funcţii

```
tip_rezultat  nume_functie (lista_arg);
{
   declaraţii – variabile locale funcţiei;

   instrucţiunile care compun funcţia;
   atribuirea unei valori ca rezultat;
}
```

Definiţia unei funcţii

- Tipul rezultatului funcţiei poate fi orice tip simplu sau structurat în afară de tablou.
- O funcţie se încheie cu instrucţiunea *return* cu sau fără parametru.
  - O funcţie poate conţine una sau mai multe instrucţiuni *return*.
- Dacă *return* conţine o expresie, valoarea acesteia este returnată către unitatea apelantă ca şi valoare a funcţiei apelate. În cazul în care expresia din *return* are un tip diferit de tipul funcţiei, va avea loc o conversie a valorii din expresie corespunzător unei atribuiri către un obiect care are acelaşi tip ca şi funcţia.

Apelul funcţiilor

- ✓ O funcţie este activată prin evaluarea apelului prin numele funcţiei într-o expresie.
- ✓ Apelul funcţiei utilizează o valoare obţinută ca rezultat al funcţiei, direct utilizabilă într-o expresie.
- ✓ Se poate apela o funcţie în funcţie (se pasează o funcţie către alta)

int func1(void);…

…

func2(func1);

Parametri formali şi parametri efectivi

- ✓ Parametrii formali apar în declaraţia funcţiei. În lista parametrilor formali se precizează ***numele*** şi ***tipul*** variabilelor utilizate de către funcţie.
- ✓ Parametrii efectivi (sau actuali) apar în apelul funcţiei. Atunci când se apelează funcţia parametrii vor fi utilizaţi efectiv cu ***anumite valori.***

f_imp.c, f_imp1.c

f_imp.c

- ✓ *Pentru rezolvarea problemei expuse se vor lua în considerare următoarele:*
  - *proba împărţirii: D = I * C + R*

  *(deîmpărţit = împărţitor * cât + rest)*
  - *descompunerea câtului în puteri ale lui doi* $C = a_n * 2^n + a_{n-1} * 2^{n-1} + \ldots + a_2 * 2^2 + a_1 * 2^1 + a_0$

f_imp.c; f_impnu.c

- ✓ *Variabilele globale 'cit' şi 'rest' pot fi declarate şi în funcţie. Variabilele globale declarate şi în funcţie sunt considerate suspendate şi inaccesibile în blocul acesteia. Deci, instrucţiunile din corpul funcţiei pot declara variabile locale sau globale care nu sunt suspendate. f_impnu.c*
- ✓ variabilele globale care pot fi utilizate şi în programul principal şi în funcţie **NU SE RECOMANDĂ**; se restrânge astfel independenţa şi gradul de generalitate al programului.

Parametri formali

- ✓ Transmiterea valorilor între programul principal (sau alte funcţii) şi o funcţie se realizează prin intermediul parametrilor formali.
  - "object declared as part of a function declaration or definition that acquires a value on entry to the function, or an identifier from the comma-separated list bounded by the parentheses immediately following the macro name in a function-like macro definition" (definiţie ANSI)
- ✓ Lista parametrilor formali are următoarea sintaxă:

(<tip_1 id1, tip_2 id2,… tip_k idk..>)

Parametri formali

- ✓ Cum parametrii formali sunt identificatori cărora li s-a asociat un tip, operaţiile care se pot efectua cu ei sunt corespunzătoare tipurilor lor;
- ✓ Descrierea tipului unui parametru formal nu se poate face în lista parametrilor, ci într-o definiţie de tip care precede declaraţia funcţiei.

Parametri formali

- ✓ Parametrii formali se comportă ca variabile locale în funcţie fiind recunoscuţi numai în interiorul ei.
- ✓ Parametrii formali sunt folosiţi pentru o descriere generală a funcţiei.
- ✓ În cazul în care în declaraţii s-a precizat un anumit număr de parametri formali, la apel trebuie să apară acelaşi număr de parametri efectivi.

Parametri actuali/efectivi

- ✓ Pot fi variabile sau expresii (valori efective)
  - "expression in the comma-separated list bounded by the parentheses in a function call expression, or a sequence of preprocessing tokens in the comma-separated list bounded by the parentheses in a function-like macro invocation" (definitia ANSI).
- ✓ Se recomandă ca tipul parametrilor formali şi al celor corespunzători efectivi/actuali să fie acelaşi.
  - În caz contrar, în limbajul C tipul parametrului efectiv este convertit automat la tipul parametrului formal.

Declaraţii de funcţii - prototip

- ✓ Se recomandă utilizarea prototipurilor, mai ales în cazul programelor complexe, deoarece pot reduce numărul de erori datorate incompatibilităţilor tipurilor sau altor caracteristici asociate funcţiei.

Cazuri generatoare de comportament nedefinit sau eronat

- ✓ În cazul în care se face un apel de funcţie şi nu există prototip, argumentele vor fi interpretate conform regulilor implicite.
- ✓ Dacă **numărul** parametrilor formali nu este acelaşi cu acela al parametrilor efectivi, comportamentul este considerat **nedefinit**.
- ✓ În cazul în care nu există un prototip, **tipul** parametrilor efectivi trebuie să fie compatibil cu acela al parametrilor formali din definiţia funcţiei, altfel comportamentul este considerat nedefinit.
- ✓ În cazul în care există un prototip şi **tipul** parametrilor efectivi nu este acelaşi cu acela al parametrilor formali din declaraţia funcţiei (adică din prototip), comportamentul este considerat nedefinit.

fcit1.c - fcit4.c

Parametri – transmiterea prin valoare şi transmiterea prin adresă

- ✓ În C toţi parametri sunt pasaţi prin valoare - funcţia apelată primeşte o copie temporară şi privată a fiecărui argument şi nu adresele lor;
- ✓ Cum argumentele funcţiei sunt pasate prin valoare, nu se vor putea modifica în mod direct valorile variabilelor din apelul funcţiei.
- ✓ Apelul prin valoare este avantajos, în general conducând la **programe mai compacte cu mai puţine variabile inutile**, deoarece argumentele pot fi tratate ca variabile locale iniţializate convenabil în funcţia apelată.

Parametri – transmiterea prin valoare şi transmiterea prin adresă

- ✓ Există şi instrumente prin care se poate **simula** un **transfer prin referinţă.**
- ✓ Pentru a transfera un obiect V ca parametru prin referinţă, se transferă un **pointer** către V; parametrul formal va fi un *pointer* către valoarea obiectului V.

- ✓ În acest mod este posibil ca o funcţie **să modifice** o variabilă în funcţia care apelat-o.
  - în apelul funcţiei se va preciza adresa variabilei a cărei valoare va fi modificată (folosind operatorul de adresare, &), iar
  - în declaraţia funcţiei argumentul va fi precizat printr-un *pointer*, referind variabila în mod indirect prin el (în corpul funcţiei se va face o dereferenţiere utilizând operatorul corespunzător, *, pentru a lucra cu variabila).

Exemple

- ✓ **f_arg.c** – calculul valorii unei funcţii pentru un argument citit de la tastatură
- ✓ **cmm1.c**
- ✓ **cmm2.c**
- ✓ **swap1.c**
- ✓ **swap2.c**

Recursivitate

- Orice funcţie poate fi chemată în ea însăşi.
- Fiecare apel al unei funcţii determină o nouă alocare a variabilelor declarate în interiorul acesteia.

Linkage

- ✓ În aplicaţii complexe se folosesc mai multe fişiere şi biblioteci; în mod uzual funcţii dintr-un fişier apelează funcţii dintr-un alt fişier.
- ✓ În C se lucrează cu 2 tipuri de entităţi: interne şi externe.
- ✓ Caracteristica intern/extern este asociată cu funcţiile: tot ceea ce este declarat în afara unei funcţii este extern, iar ceea ce este conţinut în funcţie (inclusiv parametrii formali) este considerat intern; cum nici o funcţie nu poate definită într-o altă funcţie, rezultă că funcţiile vor fi întotdeauna entităţi externe.
- ✓ La nivelul cel mai de sus, un program în C este o colecţie de entităţi externe.
- ✓ Doar acest gen de entităţi pot participa în procesul de comunicare între fişiere şi biblioteci.

Linkage

- In the set of translation units and libraries that constitutes an entire program, each declaration of a particular identifier with ***external linkage*** denotes the same object or function.
- Within one translation unit, each declaration of an identifier with ***internal linkage*** denotes the same object or function.
- Each declaration of an identifier with ***no linkage*** denotes a unique entity. (din ANSI Standard)

Linkage

Tipul legăturii	**Tipul entităţii**	**Accesibilitate**
extern	extern	pentru tot programul
intern	intern	pentru un singur fişier
fără	intern	pentru o singură funcţie

Clase de stocare

- ✓ Este posibil ca o variabilă în C să fie declarată:
  - automatic,
  - static,
  - extern,
  - registru

Clase de stocare

- ✓ Variabilele declarate ‘automatic’ sunt locale fiecărei invocări a blocului în care apar şi sunt desființate la ieşirea din bloc.
- ✓ Variabilele declarate ‘static’ sunt locale blocului, dar îşi mențin valoarea la reintrarea în bloc.
- ✓ Variabilele declarate ‘extern’ se mențin pe tot timpul execuției programului şi sunt folosite pentru comunicarea între funcții (incluzând şi funcții compilate separat).
- ✓ Variabilele declarate ‘register’ sunt memorate (dacă este posibil) în registrele rapide ale calculatorului; ca şi cele ‘automatic’ sunt locale blocului şi sunt desfiinţate la ieşirea din acesta.

Clase de stocare

- În cazul variabilelor declarate ‘extern’ şi ‘static’ iniţializarea se realizează o singură dată, pe timpul compilării.
- Pentru variabilele ‘automatic’ şi ‘register’ se realizează de fiecare dată când se intră în funcţia sau blocul care le conţine.

Sumar C10

- ✓ Tipuri. Tipuri definite de utilizator - enumerare
- ✓ Tipuri structurate
  - Tablou
  - Şir de caractere
  - Structuri